

ДИСЦИПЛИНА	Алгоритмы и структуры данных
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Текущий контроль
ПРЕПОДАВАТЕЛЬ	Преснецова Виктория Юрьевна, Яковлев Михаил Сергеевич, Дворецкий Артур Геннадьевич, Гиматдинов Дамир Маратович
СЕМЕСТР	2 семестр, 2024-2025 гг.

Рабочая тетрадь 3.

Двоичная куча. Очередь с приоритетом. Пирамидальная сортировка

Требования
1. Код должен быть оптимизирован для производительности и использования ресурсов. 2. Необходимо избегать избыточных вычислений и памяти. 3. Комментарии должны объяснять сложные участки кода и логику работы программы.

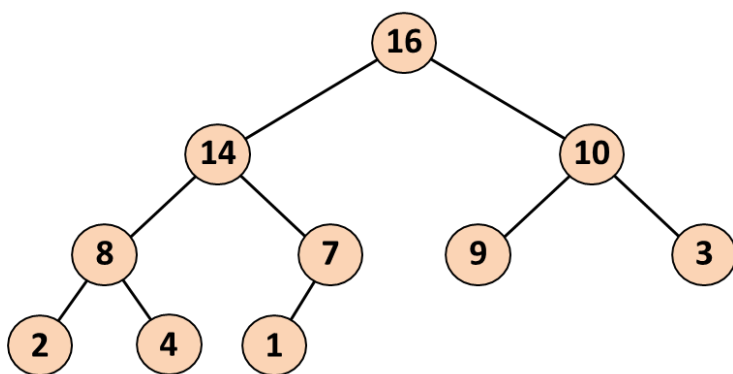
1. Двоичная куча

Теоретический материал
Куча (англ. heap) - это специализированная структура данных типа дерево, которая удовлетворяет свойству кучи: если В является узлом-потомком узла А, то $\text{ключ}(A) \geq \text{ключ}(B)$. Из этого следует, что элемент с наибольшим ключом всегда является корневым узлом кучи, поэтому иногда такие кучи называют max-кучами (в качестве альтернативы, если сравнение перевернуть, то наименьший элемент будет всегда корневым узлом, такие кучи называют min-кучами). Не существует никаких ограничений относительно того, сколько узлов-потомков имеет каждый узел кучи, хотя на практике их число обычно не более двух. Кучи обычно реализуются в виде массивов, что исключает наличие указателей между её элементами.

Над кучами обычно проводятся следующие операции:

- найти максимум или найти минимум: найти максимальный элемент в max-куче или минимальный элемент в min-куче, соответственно
- удалить максимум или удалить минимум: удалить корневой узел в max- или min-куче, соответственно
- увеличить ключ или уменьшить ключ: обновить ключ в max- или min-куче, соответственно
- добавить: добавление нового ключа в кучу.
- слияние: соединение двух куч с целью создания новой кучи, содержащей все элементы обеих исходных.

Реализация двоичной кучи на основе массива



16	14	10	8	7	9	3	2	4	1				
----	----	----	---	---	---	---	---	---	---	--	--	--	--

- Корень дерева храниться в ячейке $H[0]$ – максимальный элемент
- Индекс родителя узла i : $Parent(i) = \lfloor (i-1)/2 \rfloor$
- Индекс левого дочернего узла: $Left(i) = 2i + 1$
- Индекс правого дочернего узла: $Right(i) = 2i + 2$

Пример 1

Задача:

Реализация двоичной минимальной кучи

Решение:

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  vector<int> heap;
6
7  void heapifyUp(int index) {
8      while (index > 0) {
9          int parent = (index - 1) / 2;
10         if (heap[index] < heap[parent]) {
11             swap(heap[index], heap[parent]);
12             index = parent;
13         } else {
14             break;
15         }
16     }
17 }
18
19 // Вставка элемента в кучу
20 void insert(int value) {
21     heap.push_back(value);
22     heapifyUp(heap.size() - 1);
23 }
24
25 // Вывод кучи
26 void printHeap() {
27     for (int val : heap)
28         cout << val << " ";
29     cout << endl;
30 }
31
32 int main() {
33     insert(10);
34     insert(20);
35     insert(5);
36     insert(30);
37     insert(2);
38
39     cout << "Куча после вставки элементов: ";
40     printHeap();
41
42     return 0;
43 }

```

Ответ:

Куча после вставки элементов: 2 5 10 30 20

Задание 1 (1 балл)

Задача:

На вход программе подается массив, элементы которого расположены в произвольном порядке. Создать из массива максимальную двоичную кучу (см. для примера слайды 4 -10 лекции).

Решение:	
Ответ:	
Задание 2 (1 балл)	
Задача:	
	Создать класс «Двоичная куча» (максимальная куча). Реализовать для класса: конструктор, формирующий кучу из произвольного массива, конструктор по умолчанию (создание пустой кучи). Для класса реализовать методы: поиск максимума, удаление максимума, добавление нового элемента в кучу.
Решение:	
Ответ:	
Задание 3*	
Задача:	
	Дополните задание 2, добавив для класса метод слияния двух куч.
Решение:	
Ответ:	
Задание 4 (1 балл)	
Задача:	
	Задача о связывании канатов. Дан список (массив) канатов различной длины, которые требуется связать, по два за раз, в порядке, который приведет к наименьшим затратам. Затраты на соединение двух канатов равны сумме их длин, а общие затраты равны сумме соединения всех канатов. На выходе программа должна выдать порядок связывания канатов и суммарные затраты. Решить задачу на основе использования двоичной кучи (см. слайды 28-29 лекции)
Решение:	
Ответ:	
Задание 5*	
Задача:	

Напишите функцию, которая принимает двоичное дерево в качестве параметра и возвращает *true*, если это двоичная куча (максимальная или минимальная), и *false*, если нет.

Решение:

Ответ:

2. Очередь с приоритетом

Теоретический материал

Очередь с приоритетом - это абстрактный тип данных, описывающий структуру данных, в которой каждый фрагмент обладает определенным приоритетом. В отличие от очереди, освобождающей элементы по принципу «первым вошел, первым вышел», очередь с приоритетом обслуживает элементы в соответствии с приоритетом: сначала она удаляет данные с наивысшим приоритетом, затем данные с последующими по рангу приоритетами (или, наоборот, начинает с наименьших значений).

Очередь с приоритетом поддерживает две обязательные операции — добавить элемент и извлечь максимум (минимум). Предполагается, что для каждого элемента можно вычислить его приоритет — действительное число.

Основные методы, реализуемые очередью с приоритетом, следующие:

insert(ключ, значение) — добавляет пару (ключ, значение) в хранилище;

extract_minimum() — возвращает пару (ключ, значение) с минимальным значением ключа, удаляя её из хранилища.

При этом меньшее значение ключа соответствует более высокому приоритету.

В некоторых случаях более естественен рост ключа вместе с приоритетом. Тогда второй метод можно назвать **extract_maximum()**.

В качестве примера очереди с приоритетом можно рассмотреть список задач работника. Когда он заканчивает одну задачу, он переходит к очередной — самой приоритетной, то есть выполняет операцию извлечения максимума. Начальник добавляет задачи в список, указывая их приоритет, то есть выполняет операцию добавления элемента.

Одной из популярных реализаций очереди с приоритетом является куча.

Структура узла для очереди с приоритетом, реализованной на базе связного списка

```
struct Node {  
    int data;      // Значение элемента  
    int priority;  // Приоритет элемента (меньшее значение - более высокий  
приоритет)  
    Node* next;    // Указатель на следующий узел  
};
```

Задание 6 (1 балл)

Задача:

Реализовать очередь с приоритетом без использования кучи, используя связный список.

Решение:

Ответ:

Задание 7*

Задача:

Реализовать очередь с приоритетом на базе двоичной кучи. На основе очереди с приоритетом сделать todo list¹, для которого пользователь либо добавляет новое задание, имеющее название (одно слово) и приоритет (целое число), либо запрашивает задание с максимальным приоритетом из списка дел и делает его (помечает как выполненное, выводит за пределы очереди). Предусмотреть возможность редактирования задания.

Решение:

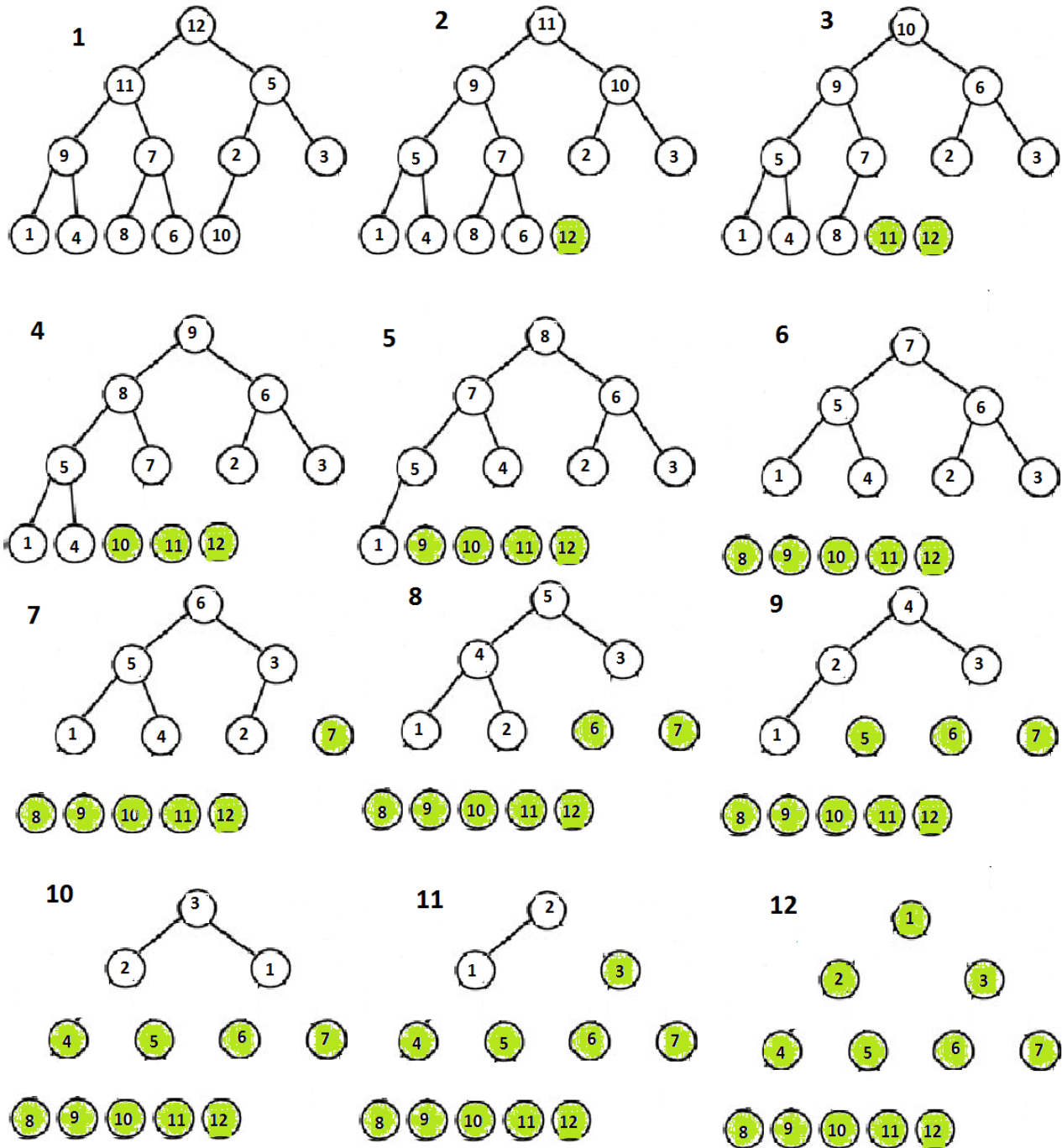
Ответ:

¹ «ToDo list» — приложение, которое позволяет пользователям добавлять, редактировать и удалять задачи, над которыми они хотят работать, а также отмечать задачи как выполненные, не удаляя их.

3. Пирамидальная сортировка

Теоретический материал

Пирамидальная сортировка (Heap Sort) – это эффективный алгоритм сортировки, основанный на двоичной куче (Binary Heap). Он работает по принципу построения кучи и поочередного извлечения максимального (или минимального) элемента.



Основные принципы работы Heap Sort

1. Преобразование массива в двоичную кучу (Heapify)

Если сортировка идет по возрастанию, то строится Мин-Куча (где каждый родитель меньше потомков).

Если сортировка идет по убыванию, то строится Min-Heap (где каждый родитель меньше потомков).

2. Извлечение максимального элемента (он всегда находится в корне Max-Heap).
 Меняем корневой элемент с последним в куче.
 Уменьшаем размер кучи и восстанавливаем свойства кучи (heapify).

3. Повторяем процесс до тех пор, пока не останется один элемент.

Задание 8 (2 балла)

Задача:	
	Реализовать алгоритм пирамидальной сортировки массива (слайды 19-25 лекции)
Решение:	
	
Ответ:	
	